UI extensibility

UI extensions SDK reference

Reference information for the UI extensions SDK, which provides utilities for app cards.

The UI extensions SDK is the foundation for adding **app card** functionality, providing an assortment of methods and utilities that enable you to:

- Access account, extension, and user context
- Perform actions like displaying alert banners
- Log custom messages for debugging
- Render the UI through UI components

Below, learn more about configuring your UI extension to access context and perform actions. For documentation on the UI components included in the SDK, check out the **UI components reference documentation**.

Registering the extension

UI extensions, like any React front-end, are written as React components. However, unlike typical React components, you must register your UI extension with HubSpot by including `hubspot.extend()` inside the component file instead of exporting it. This is not required when you create a sub-component. For cleaner code, reuse and include them inside your extension.

The `hubspot.extend()` function receives the following arguments:

- `context` : provides account, extension, and user context to the extension.
- `actions` : makes actions available to the extension.

The provided arguments can then be passed to the extension component as props.

```
// Define the extension to be run within HubSpot
hubspot.extend(({ context, actions }) => (
  <Extension
    context={context}
    sendAlert={actions.addAlert}
  />
));
// Define the Extension component, taking in context, and sendAlert as props
const Extension = ({ context, sendAlert }) => {
  ...
};
```

If your extension doesn't need to access context or perform actions, you don't need to provide these arguments to `hubspot.extend()` or the extension component.

```
hubspot.extend(() => <Extension />);
const Extension = () => {
  ...
};
```

Access context data

The `context` object contains data related to the authenticated user and HubSpot account, along with data about where the extension was loaded. You can access context data using either approach:

- **Props-based approach:** destructure `context` from the callback passed to `hubspot.extend()`, then pass it to your component as a prop.
- **Hook-based approach:** call the hook directly within your component.

The `context` object has the following fields:

HubSpot Developers

›

	`'helpdesk.sidebar'` | `'settings'` | `'home'`	
`portal.id`	Number	The ID of the HubSpot account.
`portal.timezone`	String	The account's timezone.
`portal.dataHostingLocation`	`'na1'` | `'na2'` | `'na3'` | `'ap1'` | `'eu1'`	Geographic identifier that denotes the region where the current portal is hosted. See **HubSpot Cloud Infrastructure FAQ** for more details.
`user.id`	Number	The user's ID.
`user.email`	String	The user's primary email address.
`user.emails`	Array	All of the user's associated email addresses.
`user.firstName`	String	The user's first name.
`user.lastName`	String	The user's last name.
`user.locale`	String	The user's locale.
`user.teams`	Array	An array containing information about teams that the user is assigned to. Each team object contains the `id` and `name` of the team, along with a `teammates` array that lists the IDs of other users on the team.
`user.permissions`	Array	An array of permission strings (e.g., `'integrations-management-write'`).
`variables`	Object	All of the project's **config profile** variables.

The following fields are only available in CRM extension points (`crm.record.tab` , `crm.record.sidebar` , `crm.preview` , `helpdesk.sidebar`):

Field	Type	Description
`crm.objectId`	Number	The ID of the CRM record (e.g., contact ID).

HubSpot Developers

›

`extension.appName`	String	The name of the extension's app.
`extension.cardTitle`	String	The extension's title.

Actions

Below are the actions you can perform with the SDK. You can access actions using either a props-based approach or a hook-based approach:

Props-based approach: destructure `actions` from the callback passed to `hubspot.extend()`, then pass it to your component as a prop.

Hook-based approach: call the `useExtensionActions` hook directly within your component.

> Note that some UI components include a set of actions separate from the SDK actions below, such as the **CRM action components**.

Universal actions

Supported in all extension points: `settings`, `home`, `crm.record.tab`, `crm.record.sidebar`, `crm.preview`, `helpdesk.sidebar`

Action	Description
`addAlert`	Display alert banners to the user.
`reloadPage`	Reload the current page.
`copyTextToClipboard`	Copy text to the user's clipboard.
`closeOverlay`	Close an open overlay or modal.
`openIframeModal`	Open an iframe in a modal window.

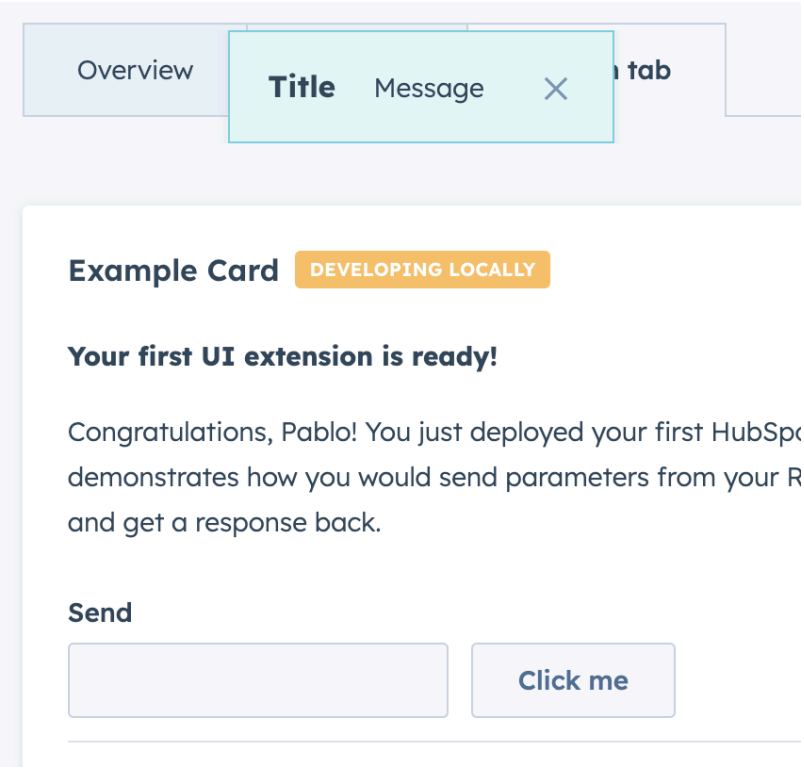
HubSpot Developers

Action	Description
`fetchCrmObjectProperties`	Fetch property values from the current CRM record.
`refreshObjectProperties`	Refresh CRM record properties on the page.
`onCrmPropertiesUpdate`	Listen for updates to CRM record properties.

While there isn't a dedicated UI component for uploading files, learn about options for **uploading files** in UI extensions.

Display alert banners

Use the `addAlert` method to send alert banners as a feedback for any actions to indicate success or failure. `addAlert` is a part of the `actions` object that can be passed to extension via `hubspot.extend`. If you instead want to render an alert within a card, check out the **Alert component**.



HubSpot Developers

HubSpot Developers

```
hubspot.extend(({actions}) => <Extension addAlert={actions.addAlert}/>);

const Extension = ({ addAlert }) => {
  const [totalUsers, setTotalUsers] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchUserData = async () => {
    try {
      setLoading(true);
      setError(null);

      const response = await hubspot.fetch('https://myExternalData.com/api/data

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const result = await response.json();
      setTotalUsers(result.stats.totalUsers);

      // Show success alert banner
      addAlert({
        title: "Data fetched successfully",
        message: `Retrieved total users from API`,
        type: "success"
      });

    } catch (err) {
      setError(err.message);
      console.error('Error fetching data:', err);

      // Show error alert banner
      addAlert({
        title: "Data Fetch Failed",
        message: `Failed to retrieve data: ${err.message}`,
        type: "danger"
      });
    } finally {
```

HubSpot Developers

```
    return (
      <>
        <LoadingSpinner />
        <Text>Fetching user data...</Text>
      </>
    );
  }

  return (
  <>
      <Text>
        Click the button to fetch the total number of registered users from the
      </Text>

      <Button onClick={fetchUserData} variant="primary">
        Fetch user data
      </Button>

      {totalUsers !== null && (
          <Text format={{ fontWeight: "demibold" }}>
            Number of users: {totalUsers}
          </Text>
      )}
</>
  );
};
```

Prop	Type	Description
`title`	String	The bolded text of the alert.
`message`	String	The main alert text.
`type`	`'info'` (default) | `'tip'` | `'success'` | `'warning'` | `'danger'`	The color of the alert.

HubSpot Developers

`warning` : a yellow alert indicating caution.

`danger` : a red alert indicating a negative outcome.

`tip` : a white alert to provide guidance.

Fetch CRM property data

There are multiple ways to fetch CRM property data via the SDK:

The `useCrmProperties` hook, which fetches properties from the current CRM record. While similar to `fetchCrmObjectProperties` , it offers automatic state management, supports property formatting, and automatically updates as properties are changed without needing to refresh.

The `fetchCrmObjectProperties` action, which fetches property data client-side at extension load time. This method is described below.

`propertiesToSend` , which can be included in your `hubspot.fetch()` functions to fetch property data on the back-end at function invocation time.

Use GraphQL to query CRM data through the `/collector/graphql` endpoint. Learn more about **querying CRM data using GraphQL**.

To make GraphQL requests, your app must include the following scopes:

```
collector.graphql_schema.read
collector.graphql_query.execute
```

fetchCrmObjectProperties

While this method is still supported, you may want to switch to using the `useCrmProperties` hook instead, which fetches properties from the current CRM record. While similar to `fetchCrmObjectProperties` , it offers automatic state management,

HubSpot Developers

›

currently displaying CRM record without having to use HubSpot's APIs. This method is a part of the `actions` object that can be passed to the extension via `hubspot.extend`. You'll first need to add the object to `objectTypes` inside the card's `.json` config file. The objects you specify in `objectTypes` will also set which CRM objects will display the extension.

```
hubspot.extend(({ actions }) => <HelloWorld fetchProperties:

const HelloWorld = ({ fetchProperties }) => {
  const [firstName, setFirstName] = useState("");
  const [lastName, setLastName] = useState("");

  useEffect(() => {
    fetchProperties(["firstname", "lastname"]).then(properties => {
      setFirstName(properties.firstname);
      setLastName(properties.lastname);
    });
  }, [fetchProperties]);

  return (
    <Text>
      Hello {firstName} {lastName}
    </Text>
  );
};
```

You can specify individual properties or fetch all properties with an asterisk:

```
fetchCrmObjectProperties("*").then(properties => console.log(properties));
```

The response for `fetchCrmObjectProperties` is formatted as:

HubSpot Developers

Refresh properties on the CRM record

Use `refreshObjectProperties` to refresh the property data on the CRM record, and any CRM data components on the record without needing to refresh the page. This includes . This method will work for the CRM objects that you include in the extension's `.json` file in the `objectTypes` array.

```
  input,
  Flex,
  hubspot
} from '@hubspot/ui-extensions';

hubspot.extend({ actions }) => (
  <Extension
    refreshObjectProperties={actions.refreshObjectProperties}
  />
));

const Extension = ({
  refreshObjectProperties,
}) => {

// Your extension logic goes here
// Refresh all properties of the object on the page
        refreshObjectProperties();
      }
    });
  };

return (
// Your extension body
)
```

Listen for property updates

Use `onCrmPropertiesUpdate` to subscribe to changes made to properties on the CRM record and run `hubspot.fetch()` functions based on those changes. This only includes changes made from within the HubSpot UI, not property updates from outside the UI, such as via APIs. This action is intended to be used like a React hook.

The full API for this method is as follows:

HubSpot Developers

```
    error?: { message: string }
    ) => void
) => void;
```

As an example, the following function subscribes to updates made to the contact's first and last name properties, then logs those properties to the console.

```
onCrmPropertiesUpdate(["firstname", "lastname"], properties
```

You can subscribe to all properties by using an asterisk.

```
onCrmPropertiesUpdate("*", properties => console.log(proper
```

To handle potential errors, pass the `error` argument to the callback.

```
onCrmPropertiesUpdate(['firstname','lastname'], (properties
   if(error) {
    console.log(error.message}
   }
   else {
     console.log(properties)
   }
})
```

Open overlays

To add another layer of UI to your extension, you can include overlays using the and components.

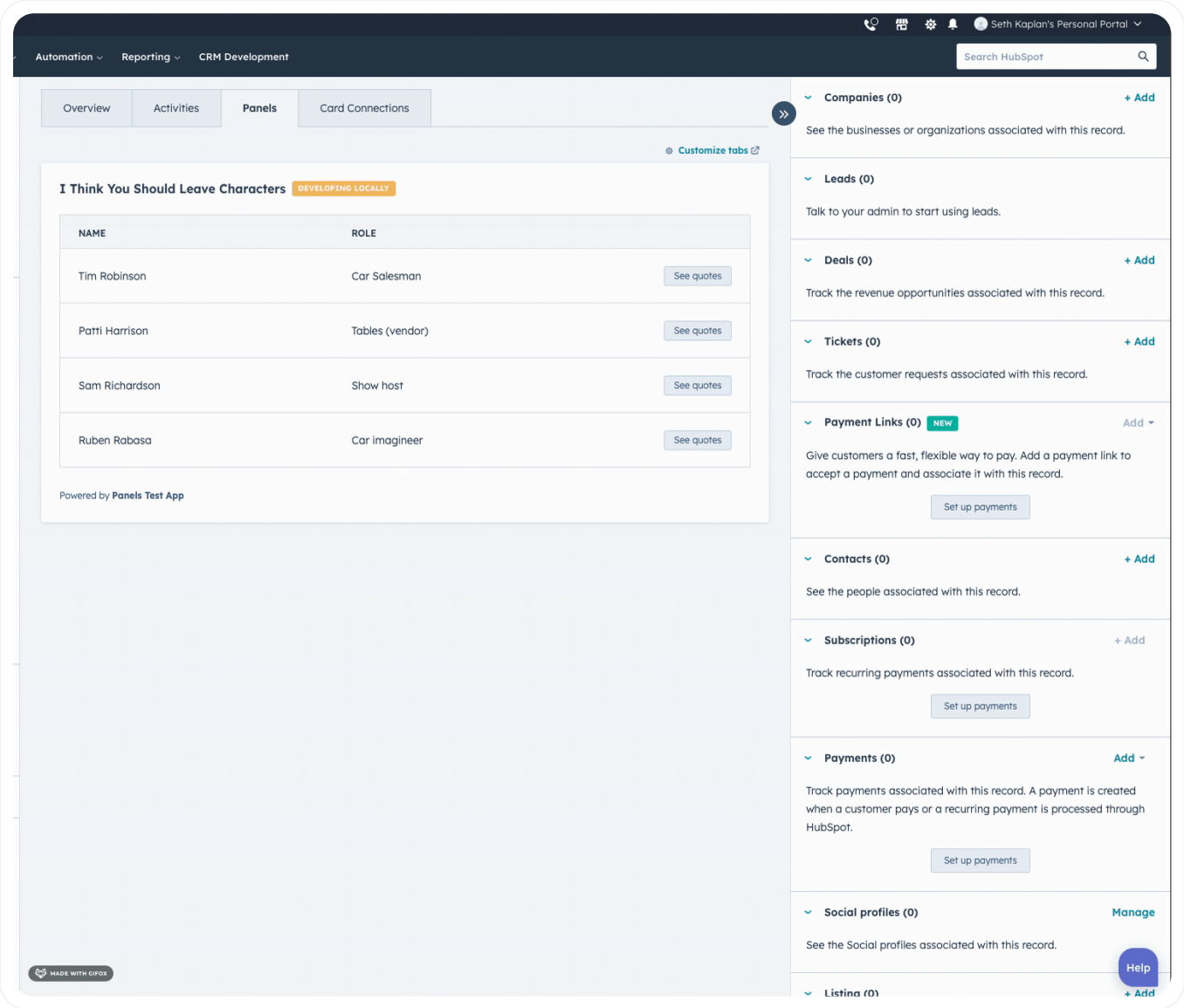
- `Modal` : a pop-up dialog box best suited for short messages and action confirmations. A `'danger'` variant is included for destructive actions, such as

HubSpot Developers

To add either type of overlay to an extension:

- Add the `overlay` prop to a **Button**, **LoadingButton**, **Link**, **Tag**, or **Image** component.
- Add the `Modal` or `Panel` component into the `overlay` prop.

Panel Modal

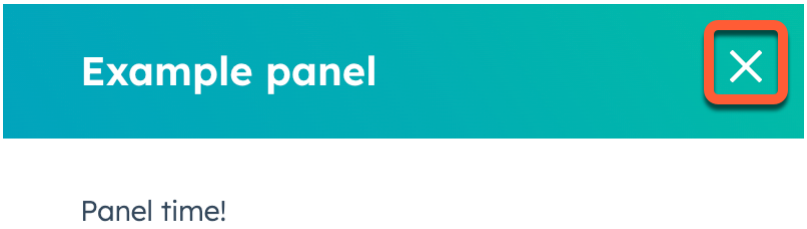


HubSpot Developers

```
const overlayExampleCard = ({ actions }) => {
  return (
    <>
```

··· See all 34 lines

By default, overlays include a close button in the top right, as shown in the example below.



> ⚠ Only one `Modal` can be open at a time per extension. Opening a `Modal` when another one is already open will cause the first one to close.
>
> A `Modal` can be opened from a `Panel`, but a `Panel` can't be opened from a `Modal`.

Open an iframe in a modal

Use the `openIframeModal` action to open an iframe in a modal window. This action accepts a payload object and an optional callback function that runs when the modal is closed.

The payload object for `openIframeModal` includes the following fields:

Field	Type	Description
`uri`	String Required	The URL to load in the iframe.
`height`	Number Required	The height of the modal in pixels.
`width`	Number Required	The width of the modal in pixels.

HubSpot Developers

›

`onClose`	Function	A callback function that runs when the modal is closed. Does not receive any parameters.

For example, the following code would result in an extension that opens an iframe on button click. The iframe is configured to contain the Wikipedia homepage with a height and width of 1000px and no padding. Upon closing the modal, a message will be logged to the console.

```
import { Link, Button, Text, Box, Flex, hubspot } from "@hubspot/ui-extensions";

hubspot.extend(({ actions }) => <Extension openIframe={actions.openIframeModal} />);

const Extension = ({ openIframe }) => {
```

··· See all 38 lines

When the user completes an action inside the iframe, the modal should close returning the user to the main page. To close the modal, the integration can use `window.postMessage` to signal that the user is done. The following messages are accepted:

- `{"action": "DONE"}` : the user has successfully completed the action.
- `{"action": "CANCEL"}` : the user has canceled the action.

```
window.top.postMessage(JSON.stringify({"action": "DONE"}),
```

Note: The domain where the action originates must match the domain of the `uri` you passed into the `openIframeModal` action. If the domains do not match, the message will be ignored.

provided by the **Clipboard: writeText() method** and follows the same requirements.

This action only works after the user has interacted with the page after loading (**transient activation**).

HubSpot Developers

›

```
function Extension({ actions }) {
  const textToCopy = `Copy me!`;

  // Use copy action on event handler
  async function handleOnClick() {
    try {
      // The function is async, make sure to await it.
      await actions.copyTextToClipboard(textToCopy);
      actions.addAlert({
        type: "success",
        message: "Text copied to clipboard.",
      });
    } catch (error) {
      // User error handling. copyTextToClipboard can fail with a `notAllowed`
      console.log(error);
      actions.addAlert({
        type: "warning",
        message: "Couldn't copy text.",
      });
    }
  }

  return (
    <Flex direction="column" gap="md">
      <TextArea label="Text" value={textToCopy} />
      <Button onClick={handleOnClick}>Copy text</Button>
    </Flex>
  );
}
```

This action should be run by explicit user interaction, otherwise the action will fail by running before the page has rendered. For example, the following implementation would fail:

HubSpot Developers

```
    /**
     * Don't run copyTextToClipboard without explicit interaction.
     * This will fail because the action will run before the page
     * has rendered.
     */
    async function badStuff() {
      try {
        await actions.copyTextToClipboard(textToCopyWithoutUserPermission);
        actions.addAlert({
          type: 'success',
          message: 'text copied to clipboard',
        });
      } catch (error) {
        console.log(error);
        actions.addAlert({
          type: 'warning',
          message: "can't copy value",
        });
      }
    }

    badStuff();
  }, []);
```

Reload page

Use the `reloadPage` action to reload the current page. This action can be accessed through the actions argument (`actions.reloadPage`).

HubSpot Developers

```
function Extension({ actions }) {
  return (
    <Button onClick={() => actions.reloadPage()}>
      Reload page
    </Button>
  );
}
```

Upload files

While there is no for uploading files, there are a few ways you can upload files:

, then use a to display and manage the property from CRM records. You can upload up to 10 files per file property, and file uploaded via file properties have the same as files uploaded to the file manager.

Include an in the extension that loads an upload page, then upload files through the iframe.

Hooks

The SDK provides hooks to simplify accessing context, performing actions, and fetching CRM data within UI extensions. These hooks are optimized to prevent unnecessary re-renders and automatically clean up resources when components unmount. You can pass inline arrays and objects to the hooks directly, as memoization is not required.

Both approaches provide identical functionality, so the choice is a matter of preference. As a general guideline:

Hook-based approach: cleaner component APIs, no prop drilling.

Props-based approach: explicit dependency injection.

Either way, the data is the same, as shown in the code example below.

HubSpot Developers

```
function MyExtension({ context, actions }) {
  return (
```

··· See all 25 lines

Universal hooks

These hooks are available in all extension points (`crm.record.tab` , `crm.record.sidebar` , `crm.preview` , `helpdesk.sidebar` , `settings` , and `home`).

```
import {
  useExtensionApi,
  // OR
  useExtensionContext,
  useExtensionActions
} from "@hubspot/ui-extensions";
```

- `useExtensionApi` : access both context and actions from a single hook.
- `useExtensionContext` : access contextual information about the extension environment.
- `useExtensionActions` : access actions that can be performed within HubSpot.

useExtensionApi

The `useExtensionApi` hook provides access to available actions and contextual information from a single hook, for extension components that need access to both actions and context.

Otherwise, it's best practice to use the more specific `useExtensionActions` or `useExtensionContext` , depending on your use case.

For a complete list of available context properties, see **Access context data**.

HubSpot Developers

containing the user's first name (context) when a button is clicked.

Basic usage Typed usage

```
import { Button, hubspot, useExtensionApi } from '@hubspot/

hubspot.extend(() => <MyExtension />);

function MyExtension() {
  const { actions, context } = useExtensionApi();

  return (
    <Button onClick={() => actions.addAlert({ message: `Hello ${context.user.fi
      Click me
    </Button>
  );
}
```

useExtensionContext

The `useExtensionContext` hook provides access to contextual information about the current extension environment, including location and other relevant data.

For a complete list of available context properties, see **Access context data**.

The following example accesses the current extension location and renders it in a `Text` component:

Basic usage Typed usage

HubSpot Developers

›

```
function MyExtension() {
  const context = useExtensionContext();

  return (
    <Text>Current location: {context.location}</Text>
  );
}
```

useExtensionActions

The `useExtensionActions` hook provides access to various actions that can be performed within the HubSpot interface. It's a generic hook that can be typed with specific extension point locations for better TypeScript support.

For a complete list of available actions, see **Actions**. The actions available depend on the extension point location.

The following example displays an alert when a button is clicked:

Basic usage Typed usage

HubSpot Developers

```
function MyExtension() {
  const { addAlert } = useExtensionActions();

  return (
    <Button onClick={() => addAlert({ message: "Action completed!" })}>
      Click me
    </Button>
  );
}
```

CRM-specific hooks

These hooks are only available in CRM extension points (`crm.record.tab` , `crm.record.sidebar` , `crm.preview` , and `helpdesk.sidebar`).

- `useCrmProperties` : fetch properties from the current CRM record
- `useAssociations` : fetch associated CRM records

```
import {
  useCrmProperties,
  useAssociations
} from "@hubspot/ui-extensions/crm";
```

useCrmProperties

The `useCrmProperties` hook fetches properties from the current CRM record with optional formatting. It accepts an array of properties to fetch, along with an optional object to format the returned data.

Call Response Example usage

HubSpot Developers

```
    date: {
      format: "MM-DD-YYYY",
      relative: false,
    },
    dateTime: {
      format: "MM-DD-YYYY hh:mm",
      relative: false,
    },
    currency: {
      addSymbol: true,
    },
  },
});
```

`propertiesToFormat` `'all' | array` **required**

Either `'all'` or an array of property names to format.

`formattingOptions` `object`

Contains formatting options for the values returned from date, datetime, and currency properties.

The `date` and `dateTime` objects can include `format` and `relative` subfields:

- `format` (string): a date or datetime string like `MM-DD-YYYY` or `MM-DD-YYYY:mm:ss`. Supports .
- `relative` (boolean): set to `true` to display the amount of time passed since the returned value (e.g., `(1 day ago)` or `(1 hour ago)`).

The `currency` object can include `addSymbol` (boolean), which sets whether the currency symbol should display with the number. Set to `true` to display the currency symbol.

> ⚠ Formatting is applied based on property type (`date` , `datetime` , `currency`) rather than content. For example, `date` formatting will not apply to a string type property containing a date value.

HubSpot Developers

›

data.

Call Response Example usage

```
useAssociations(
  {
    // Object type ID to fetch associations for
    toObjectType: "0-1",
    // Optional properties to fetch from associated records
    properties: ["firstname", "lastname", "email", "phone"],
    // Optional pagination settings
    pageLength: 25,
  },
  // Optional formatting configuration (same as useCrmProperties)
  {
    propertiesToFormat: "all",
    formattingOptions: {
      date: {
        format: "MM-DD-YYYY",
        relative: false,
      },
      dateTime: {
        format: "MM-DD-YYYY hh:mm",
        relative: false,
      },
      currency: {
        addSymbol: true,
      },
    },
  }
);
```

config object required

HubSpot Developers

›

`propertiesToFormat` `'all' | array` `required`

Either `'all'` or an array of property names to format.

`formattingOptions` `object`

Contains formatting options for the values returned from date, datetime, and currency properties.

The `date` and `dateTime` objects can include `format` and `relative` subfields:

`format` (string): a date or datetime string like `MM-DD-YYYY` or `MM-DD-YYYY:mm:ss` . Supports **standard date time string formats**.

`relative` (boolean): set to `true` to display the amount of time passed since the returned value (e.g., `(1 day ago)` or `(1 hour ago)`).

The `currency` object can include `addSymbol` (boolean), which sets whether the currency symbol should display with the number. Set to `true` to display the currency symbol.

Best practices

Always use TypeScript generics

```
// ✅ Good - Typed for better IntelliSense and type safety
const actions = useExtensionActions<'crm.record.tab'>();
const context = useExtensionContext<'crm.record.tab'>();

// ❌ Avoid - Less type safety and IntelliSense
const actions = useExtensionActions();
const context = useExtensionContext();
```

Extract hook calls to component level

HubSpot Developers

```
  const handleClick = () => {
    addAlert({ message: `Action from ${context.location}` });
  };

  return <Button onClick={handleClick}>Click me</Button>;
}

// ❌ Avoid - Don't call hooks in event handlers
function MyExtension() {
  const handleClick = () => {
    const { addAlert } = useExtensionActions(); // Wrong!
    addAlert({ message: "Hello" });
  };

  return <Button onClick={handleClick}>Click me</Button>;
}
```

Send custom log messages for debugging

Using `logger` methods, you can send custom log messages to HubSpot for more in-depth troubleshooting of deployed extensions. Custom log messages will appear in the .

The following methods are available:

- `logger.info`
- `logger.debug`
- `logger.warn`
- `logger.error`

Each method accepts a single string argument.

For example, the following extension code includes few different log messages to help better identify where an error has occurred:

HubSpot Developers

```
logger.warn("Warning in the middle tab, before my extension");

hubspot.extend(({ context }) => <MiddleTabLogging context={context} />);
```

··· See all 67 lines

When an extension fails to load on a CRM record, an error message will display. This error message will contain a trace ID, which you can copy.

> There was a problem displaying this content. Try **refreshing the page**. If this problem continues, contact the app developer for support with this trace ID: c6c56f65-d1b6-41ee-8701-2a6c04f4f170 .

Using that trace ID, you can then locate the custom log messages **within the extension's logs**.

Notes and limitations

- Custom log messages are not sent while in local development mode. They are logged to the browser console instead.
- All logs are sent as batches with a maximum of 100 logs per batch.
- Each HubSpot account is rate limited to 1,000 logs per minute. After exceeding that limit, all logging is stopped until the page is reloaded.
- The logger will queue a maximum of 10,000 pending messages. Any subsequent logs will be dropped until the queue is below the maximum.
- Queued logs are processed at a rate of five seconds per log batch.
- Queued logs are dropped when the page or is refreshed or closed.

Last modified on February 19, 2026

HubSpot Developers

Fetching data for UI extensions
Previous

Share code between your extensions using npm workspaces
Next

HubSpot Developers
Copyright © 2026 HubSpot, Inc
Privacy Policy
Security
Website Accessibility
Manage Cookies